

EE345 Mod4-Lec1: Classification via Logistic Regression

[VMLS, 14]

Classification

- **So far...**: goal was to predict a real-valued outcome y from some data x
- **Module 4** (Classification): the outcome y takes on only a finite number of values, and hence is sometimes called a **label**, or in statistics, a **categorical**.

Binary Classification

- As before, the goal is to predict the outcome y from the feature vector x .
- The classifier f is a function that takes in a feature vector x and outputs a prediction $f(x)$.
- But now the outcome y takes on only two values, typically $\{-1, +1\}$ or $\{0, 1\}$.

From Regression to Classification

- So far: **least squares** for regression problems

$$y \approx \theta^\top x, \quad y \in \mathbb{R}.$$

- Now: **binary classification**

$$y \in \{0, 1\}, \quad \text{e.g., spam vs. not spam, click vs. no click.}$$

- Problem: a linear model $\theta^\top x$ can be any real number, but we want

$$\text{score}(x) \in [0, 1] \quad \text{OR} \quad p(y = 1 \mid x) \in [0, 1].$$

- Idea: keep a linear score, then pass it through a *squashing* function to get a probability.

Classification Examples

- **Email spam detection.**

- ▶ *Feature vector:* $x \in \mathbb{R}^n$ contains features of an email message like word counts etc.
- ▶ *Outcome:* $y = +1$ if an email represented by feature vector x is **SPAM** and -1 otherwise.

- **Fraud detection.**

- ▶ *Feature vector:* $x \in \mathbb{R}^n$ contains features associated with a credit card user such as average monthly spending, median prices of purchases over last week, etc.
- ▶ *Outcome:* $y = +1$ for **fraudulent transactions**, and -1 otherwise.

- **Document Classification.**

- ▶ *Feature vector:* $x \in \mathbb{R}^n$ is a word count (or histogram) vector for a document
- ▶ *Outcome:* $y = +1$ if the document has some topic (e.g., politics) and -1 otherwise

Two Linear Approaches to Binary Classification

- We consider $y \in \{-1, +1\}$ and a linear score:

$$s(x) = \theta^\top x.$$

- Two ways to turn scores into predictions:
 - ▶ **Least Squares Classification:** choose θ to make $s(x_i)$ close to y_i .
 - ▶ **Logistic Regression:** choose θ to model $p(y_i = 1 \mid x_i)$ via logistic function.

Method 1: Least Squares Classification

Least Squares Classifier

- Least squares classifier: this is a simple method that works well in many cases
- do ordinary real-valued least squares fitting of the outcome, ignoring that $y \in \{-1, +1\}$
- i.e., choose basis functions $\phi_1, \dots, \phi_p$ and solve

$$\min_{\theta \in \mathbb{R}^p} \left\{ \sum_{i=1}^N (y_i - \tilde{f}(x_i))^2 \mid \tilde{f}(x_i) = \theta_1 \phi_1(x_i) + \dots + \theta_p \phi_p(x_i), \ i = 1, \dots, N \right\}$$

- $\tilde{f}$ is the least squares fit over our data
- **final classifier (least squares classifier):**

$$\hat{f}(x) = \text{sign}(\tilde{f}(x)) \quad \text{where} \quad \text{sign}(a) = \begin{cases} +1, & a \geq 0 \\ -1, & a < 0 \end{cases}$$

- e.g., $\tilde{f}(x) = x^\top \beta + v$ —i.e., a regression model so that $f(x) = \text{sign}(x^\top \beta + v)$

Intuition for Least Squares Classifier

- The value $\tilde{f}(x)$ is a number "near" $+1$ when $y = +1$ and near -1 when $y = -1$
- Forced to guess one of the two possible outcomes, $\text{sign}(\tilde{f}(x))$ is a good choice—it is the nearest neighbor of $\tilde{f}(x)$ among $\{-1, +1\}$
- $\tilde{f}(x)$ also tells us our confidence in our assignment

Least Squares Classification

- If we pick $\phi_1(x) = 1$ and $\phi_2(x) = x$, then the problem is

$$\min_{\theta} \|X\theta - y\|_2^2, \quad \text{where } y_i \in \{-1, +1\}.$$

- Closed-form solution:

$$\hat{\theta}_{\text{LS}} = (X^{\top} X)^{-1} X^{\top} y.$$

- Prediction rule:

$$\hat{y}(x) = \text{sign}(\hat{\theta}_{\text{LS}}^{\top} x) = \begin{cases} +1, & \hat{\theta}_{\text{LS}}^{\top} x \geq 0, \\ -1, & \hat{\theta}_{\text{LS}}^{\top} x < 0. \end{cases}$$

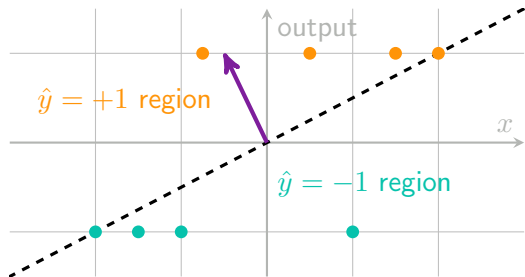
- Pros:

- ▶ Simple, fast, closed-form.
- ▶ Works surprisingly well when classes are linearly separable.

- Cons:

- ▶ Squared error is not made for classification.
- ▶ Large errors are overly penalized (outliers).
- ▶ Predicted scores are not probabilities.

Least Squares Classifier with the Sign Function



- Map labels to $\{-1, +1\}$ and fit:

$$\hat{\theta} = \arg \min_{\theta} \sum_i (y_i - \theta_0 - \theta_1 x_i)^2.$$

- Prediction is a **real-valued** output

$$f(x) = \theta_0 + \theta_1 x.$$

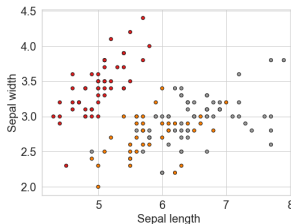
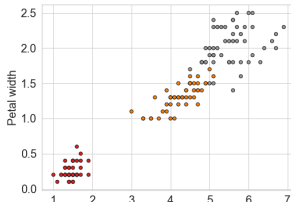
- Turn it into a classifier with

$$\hat{y}(x) = \text{sign}(f(x)).$$

- Decision boundary: where $f(x) = 0$ (dashed line).
- Least squares tries to fit the *numeric targets*, then we *post-process* with a sign; this is why it can behave poorly for classification.

Example: Predicting Iris Type

- Iris data set: classical ML data set
- Three types of iris:
 - ▶ setosa, versicolour, virginica
- Four features:
 - ▶ x_1 sepal length [cm], x_2 sepal width [cm]
 - ▶ x_3 petal length [cm], x_4 petal width [cm]
- 50 samples of each type
- Goal: build classifier to detect if iris is virginica or not



Iris Data Set Example: Confusion Matrix

- data matrix

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & x_3^{(1)} & x_4^{(1)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} & x_3^{(N)} & x_4^{(N)} \end{bmatrix}$$

- labels: $y = (\underbrace{-1, \dots, -1}_{100 \text{ times}}, \underbrace{+1, \dots, +1}_{50 \text{ times}})$

- $\min_x \|X\theta - y\|_2^2$

- **Solution:**

$$\hat{x} = (X^\top X)^{-1} X^\top y, \quad \hat{y}_i = \text{sign}(x_i^\top \hat{\theta}).$$

Method 2: Logistic Regression

Logistic Regression: Fixing the Problems

Instead of predicting y directly, model the probability:

$$p(y = 1 \mid x) = \sigma(\theta^\top x) = \frac{1}{1 + e^{-\theta^\top x}}.$$

Interpretation:

- Output is always in $[0, 1]$.
- Smooth transition between classes.

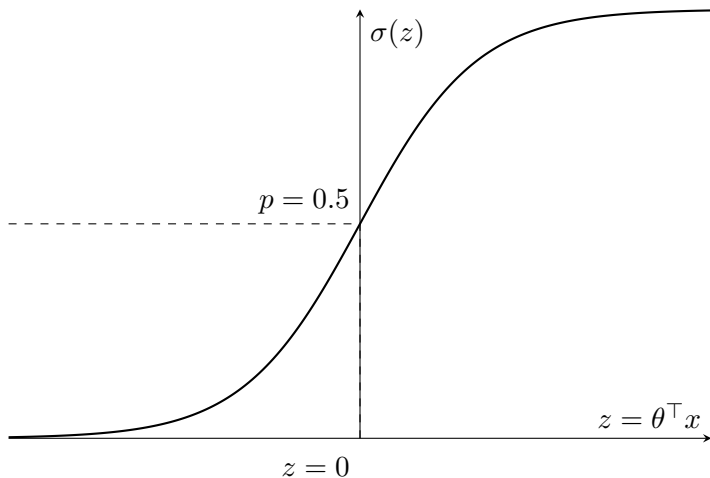
Decision Boundary

Because the sigmoid is monotone:

$$p(y = 1 \mid x) = 0.5 \iff \theta^\top x = 0.$$

Same boundary as linear regression classification, BUT better calibrated probabilities.

Sigmoid



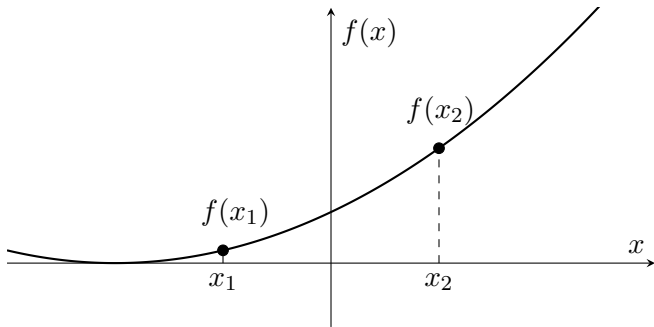
The linear score $z = \theta^\top x$ is mapped through the sigmoid to a probability $\sigma(z) \in (0, 1)$;
 $z = 0$ corresponds to $p = 0.5$.

Sigmoid Properties: Monotone functions are order-preserving

A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is **monotone increasing** if

$$x_1 < x_2 \implies f(x_1) \leq f(x_2).$$

- Intuition: as you move to the right on the x -axis, the function value never goes down.



Checking Monotonicity Using the Derivative

Monotone increasing: if $f \in C^1$ (i.e. continuous and differentiable)

$$f'(x) \geq 0 \quad \text{for all } x \quad \Longleftrightarrow \quad x_1 < x_2 \Rightarrow f(x_1) \leq f(x_2).$$

Reason: For any $x_1 < x_2$,

$$f(x_2) - f(x_1) = \int_{x_1}^{x_2} f'(t) dt.$$

If $f'(t) \geq 0$ on $[x_1, x_2]$, then the integral is ≥ 0 .

Strictly monotone increasing: $f'(x) > 0$ for all x

Monotone decreasing: $f'(x) \leq 0$ for all x

Derivative Test: Necessary and Sufficient (for C^1)

Assume: $f : \mathbb{R} \rightarrow \mathbb{R}$ is continuously differentiable (i.e., $f \in C^1$).

Sufficient:

$$f'(x) \geq 0 \quad \forall x \quad \implies \quad f \text{ is monotone non-decreasing.}$$

Necessary: If f is monotone non-decreasing, then for every x ,

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \geq 0.$$

Conclusion: For C^1 (and therefore C^2) functions,

$$f \text{ monotone increasing} \quad \iff \quad f'(x) \geq 0 \quad \forall x.$$

Note: Without differentiability, monotonicity does *not* require f' to exist.

Monotonicity of the Sigmoid

The logistic (sigmoid) function is

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

- $0 < \sigma(z) < 1 \quad \forall z \implies 0 < \sigma'(z) < 1 \quad \forall z.$
- So σ is **strictly monotone increasing**:

$$z_1 < z_2 \implies \sigma(z_1) < \sigma(z_2).$$

Consequences for classification:

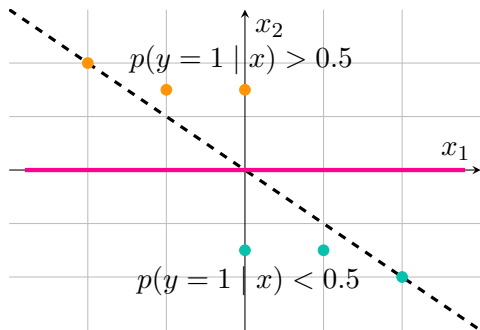
- Threshold on probability:

$$p(y = 1 \mid x) = \sigma(\theta^\top x) \geq 0.5 \quad \iff \quad \theta^\top x \geq 0.$$

- Thus the decision boundary for $\hat{y} = \mathbf{1}\{p \geq 0.5\}$ is the hyperplane $\theta^\top x = 0$.

Decision Boundary for Logistic Regression

$$p(y = 1 \mid x) = \sigma(\theta^\top x).$$



- Because σ is **monotone increasing**,

$$p(y = 1 \mid x) = 0.5 \iff \theta^\top x = 0.$$

- So the decision rule

$$\hat{y}(x) = \mathbf{1}\{p(y = 1 \mid x) \geq 0.5\}$$

is equivalent to

$$\hat{y}(x) = \mathbf{1}\{\theta^\top x \geq 0\}.$$

- Geometrically: a **linear boundary** (hyperplane) separating the $p > 0.5$ and $p < 0.5$ regions.

Why Can Sigmoid Outputs Be Interpreted as Probabilities?

Odds and Log-Odds

Odds

For a probability $p \in (0, 1)$, the *odds* of the event are

$$\text{odds} = \frac{p}{1-p}.$$

Log-odds (logit)

$$\log\left(\frac{p}{1-p}\right).$$

In logistic regression,

$$p(x) := p(y = 1 \mid x) = \sigma(\theta^\top x) \implies \log\left(\frac{p(x)}{1-p(x)}\right) = \theta^\top x.$$

So the **log-odds** are a *linear* function of x .

Interpreting Coefficients via Odds Ratios

Consider a single feature x and model

$$\log \left(\frac{p(y = 1 \mid x)}{1 - p(y = 1 \mid x)} \right) = \theta_0 + \theta_1 x \quad \implies \quad \text{log-odds} = \theta_0 + \theta_1 x.$$

- For x and $x + \Delta$, the log-odds differ by

$$(\theta_0 + \theta_1(x + \Delta)) - (\theta_0 + \theta_1 x) = \theta_1 \Delta.$$

- Thus the **odds** are multiplied by $e^{\theta_1 \Delta}$:

$$\frac{\text{odds at } x + \Delta}{\text{odds at } x} = e^{\theta_1 \Delta}.$$

Interpretation: A one-unit increase in x multiplies the odds of $y = 1$ by e^{θ_1} (holding other features fixed).

Why Can Sigmoid Outputs Be Interpreted as Probabilities?

- Logistic regression assumes a probabilistic model:

$$y_i \sim \text{Bernoulli}(p_i), \quad p_i = P(y_i = 1 \mid x_i, \theta).$$

- We model the **log-odds** as linear:

$$\log \frac{p_i}{1 - p_i} = \theta^\top x_i.$$

- Solving for p_i gives the sigmoid:

$$p_i = \sigma(\theta^\top x_i) = \frac{1}{1 + e^{-\theta^\top x_i}}.$$

- Therefore $\sigma(\theta^\top x)$ is the **model's estimate** of the probability that $y = 1$.
- **Calibration:** These probabilities may or may not be well-calibrated depending on the dataset and training.

Fitting the Logistic Regression Model

Summary so far: Logistic Regression

- Model a probability:

$$p(y = 1 \mid x) = \sigma(\theta^\top x) = \frac{1}{1 + e^{-\theta^\top x}}.$$

- "Maximum likelihood estimation":

$$\min_{\theta} - \sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right) = \min_{\theta} J(\theta).$$

- Gradient: **we claim that**

$$\nabla_{\theta} J(\theta) = X^\top (p - y),$$

- Prediction rule:

$$\hat{y}(x) = \begin{cases} 1, & p(x) \geq 0.5, \\ 0, & p(x) < 0.5. \end{cases}$$

- Outputs calibrated probabilities.

Likelihood and Loss for Logistic Regression

- Data: (x_i, y_i) for $i = 1, \dots, n$, with $y_i \in \{0, 1\}$.
- Model:

$$p_i := p(y_i = 1 \mid x_i) = \sigma(\theta^\top x_i).$$

- Likelihood (under independence):

$$L(\theta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}.$$

- Log-likelihood:

$$\ell(\theta) = \sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right).$$

- We **maximize** $\ell(\theta)$, or equivalently **minimize** the negative log-likelihood:

$$J(\theta) = -\ell(\theta) = -\sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right).$$

Loss Function: Negative Log-Likelihood or Cross-Entropy Loss

Maximum likelihood $\implies$ minimize logistic loss:

$$\ell(\theta) = - \sum_{i=1}^n \left[y_i \log(\sigma(\theta^\top x_i)) + (1 - y_i) \log(1 - \sigma(\theta^\top x_i)) \right].$$

Properties:

- Convex in θ .
- No closed form $\longrightarrow$ use gradient descent.

Gradient of the Logistic Loss

- Recall $p_i = \sigma(\theta^\top x_i)$ and

$$J(\theta) = - \sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right).$$

- Using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, one can show

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n (p_i - y_i) x_i.$$

- In matrix notation: let $X \in \mathbb{R}^{n \times d}$ have rows $x_i^\top$, $p \in \mathbb{R}^n$ have entries p_i , and $y \in \mathbb{R}^n$ have entries y_i . Then

$$\nabla_{\theta} J(\theta) = X^\top (p - y).$$

Gradient of the Logistic Loss: Detailed Derivation (1)

Recall:

$$J(\theta) = - \sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right), \quad p_i = \sigma(z_i), \quad z_i = \theta^\top x_i.$$

We differentiate w.r.t. θ using the chain rule. First, focus on a single i :

$$J_i(\theta) = - \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right).$$

Compute the gradient:

$$\nabla_{\theta} J_i(\theta) = - \left[y_i \frac{1}{p_i} \nabla_{\theta} p_i + (1 - y_i) \frac{1}{1 - p_i} \nabla_{\theta} (1 - p_i) \right].$$

Gradient of the Logistic Loss: Detailed Derivation (2)

Note that $\nabla_{\theta}(1 - p_i) = -\nabla_{\theta}p_i$, so

$$\nabla_{\theta}J_i(\theta) = - \left[y_i \frac{1}{p_i} \nabla_{\theta}p_i - (1 - y_i) \frac{1}{1 - p_i} \nabla_{\theta}p_i \right].$$

Factor $\nabla_{\theta}p_i$:

$$\nabla_{\theta}J_i(\theta) = - \left[\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} \right] \nabla_{\theta}p_i.$$

Gradient of the Logistic Loss: Detailed Derivation (3)

We still need $\nabla_{\theta} p_i$ and to simplify the coefficient.

Step 1: $p_i = \sigma(z_i)$ with $z_i = \theta^{\top} x_i$.

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad \Rightarrow \quad \frac{\partial p_i}{\partial z_i} = p_i(1 - p_i).$$

By the chain rule,

$$\nabla_{\theta} p_i = \frac{\partial p_i}{\partial z_i} \nabla_{\theta} z_i = p_i(1 - p_i) x_i.$$

Gradient of the Logistic Loss: Detailed Derivation (4)

Step 2: simplify the bracket:

$$\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i} = \frac{y_i(1 - p_i) - (1 - y_i)p_i}{p_i(1 - p_i)} = \frac{y_i - y_i p_i - p_i + y_i p_i}{p_i(1 - p_i)} = \frac{y_i - p_i}{p_i(1 - p_i)}.$$

Thus

$$\nabla_{\theta} J_i(\theta) = -\frac{y_i - p_i}{p_i(1 - p_i)} \cdot p_i(1 - p_i) x_i = -(y_i - p_i) x_i = (p_i - y_i) x_i.$$

Total gradient:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} J_i(\theta) = \sum_{i=1}^n (p_i - y_i) x_i.$$

In matrix notation, with $X \in \mathbb{R}^{n \times d}$, $p, y \in \mathbb{R}^n$:

$$\nabla_{\theta} J(\theta) = X^{\top} (p - y).$$

Gradient Descent for Logistic Regression

- We minimize $J(\theta)$ using gradient descent:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} J(\theta^{(t)}),$$

where $\eta > 0$ is the step size.

- Plug in the gradient:

$$\theta^{(t+1)} = \theta^{(t)} - \eta X^{\top} (p^{(t)} - y),$$

where $p_i^{(t)} = \sigma((\theta^{(t)})^{\top} x_i)$.

- Just like least squares:
 - ▶ Linear model: $\theta^{\top} x$.
 - ▶ Different loss: logistic (negative log-likelihood) instead of squared error.

Regularized Logistic Regression (Optional)

- To control overfitting, we can add an ℓ_2 penalty:

$$J_\lambda(\theta) = - \sum_{i=1}^n \left(y_i \log p_i + (1 - y_i) \log(1 - p_i) \right) + \lambda \|\theta\|_2^2.$$

- Gradient becomes

$$\nabla_\theta J_\lambda(\theta) = X^\top (p - y) + 2\lambda\theta.$$

- Gradient descent update:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \left(X^\top (p^{(t)} - y) + 2\lambda\theta^{(t)} \right).$$

- Same idea as ridge regression, but with logistic loss.

Prediction Errors: Model Performance Metrics

Prediction Errors

For a given data point (x, y) with predicted outcome $\hat{y} = f(x)$, there are four possibilities:

1. *True Positive*: $y = +1$ and $\hat{y} = +1$ [correct prediction]
2. *True Negative*: $y = -1$ and $\hat{y} = -1$ [correct prediction]
3. *False Positive*: $y = -1$ and $\hat{y} = +1$ [incorrect prediction, type I error]
4. *False Negative*: $y = +1$ and $\hat{y} = -1$ [incorrect prediction, type II error]

Error Rates

Consider data set $(x_1, \dots, x_n)$, $(y_1, \dots, y_n)$ and model f .

- N_p : number of outcomes correctly classified $y = +1$
- N_n : number of outcomes correctly classified $y = -1$
- N_{tp} : number of true positives
- N_{fp} : number of false positives (Type I error)
- N_{fn} : number of false negatives (Type II error)
- N_{tn} : number of true negatives (correctly classified $y = -1$)

Error Rates

- *Error rate*: total number of errors of both kinds divided by the number of examples

$$\frac{N_{\text{fp}} + N_{\text{fn}}}{N}$$

- *True positive rate* (sensitivity/recall rate):

$$\frac{N_{\text{tp}}}{N_{\text{p}}} \quad \text{where } N_{\text{p}} = \# \text{ outcomes with a true positive label } y = +1$$

- *False positive rate* (false alarm rate):

$$\frac{N_{\text{fp}}}{N_{\text{n}}} \quad \text{where } N_{\text{n}} = \# \text{ outcomes with a true negative label } y = -1$$

Error Rates

- *True negative rate* (specificity):

$$1 - \frac{N_{\text{fp}}}{N_{\text{n}}} = \frac{N_{\text{tn}}}{N_{\text{n}}} \quad \text{fraction of data with } y = -1 \text{ correctly classified as } \hat{y} = -1$$

- *Precision*:

$$\frac{N_{\text{tp}}}{N_{\text{tp}} + N_{\text{fp}}} \quad \text{i.e., fraction of true predictions that are correct.}$$

Confusion Matrix

- *Good classifier*: small (near zero) error rate and false positive rate, and high (near one) true positive rate, true negative rate, and precision.
- Which of these metrics is more important depends on the particular application.

outcome	prediction		total
	$\hat{y} = +1$	$\hat{y} = -1$	
$y = +1$	N_{tp}	N_{fn}	N_{p}
$y = -1$	N_{fp}	N_{tn}	N_{n}
all	$N_{\text{tp}} + N_{\text{fp}}$	$N_{\text{fn}} + N_{\text{tn}}$	N

Iris Data Set Example: Confusion Matrix

- Precision:

$$\frac{N_{tp}}{N_{tp} + N_{fp}}$$

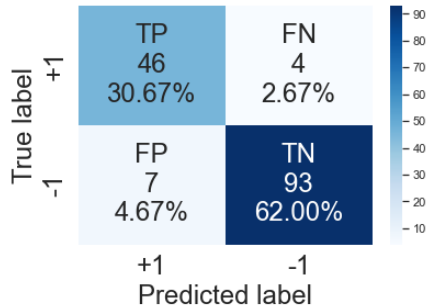
- Accuracy:

$$\frac{N_{tp} + N_{tn}}{N}$$

- F1-score is the harmonic mean of precision (P) and recall (R):

$$\frac{2PR}{P+R}$$

- ▶ recall: $N_{tp}/(N_{tp} + N_{fn})$
- ▶ want it near 1



Accuracy=0.927; Precision=0.959
Recall=0.930; F1 Score=0.944

Receiver Operating Characteristic (ROC) Curves

Many classifiers output a **score** or **probability**:

$$s_i = p(y_i = 1 \mid x_i).$$

To turn scores into predictions, choose a **threshold** t and predict:

$$\hat{y}_i(t) = \mathbf{1}\{s_i \geq t\}.$$

For each threshold t , we can compute:

$$\text{TPR}(t) = \frac{TP(t)}{TP(t) + FN(t)} \quad (\text{True Positive Rate})$$

$$\text{FPR}(t) = \frac{FP(t)}{FP(t) + TN(t)} \quad (\text{False Positive Rate}).$$

ROC Curve

Receiver Operating Characteristic (ROC) curve:

$$\text{ROC} = \{(\text{FPR}(t), \text{TPR}(t)) : t \in (-\infty, +\infty)\}.$$

It plots the tradeoff between false positives and true positives across all possible thresholds. **ROC curve** shows how TPR changes as we vary the threshold t . Useful when:

- classes are imbalanced,
- the cost of FP vs FN differs,
- we want threshold-independent comparison.

Constructing the ROC Curve

Step 1: Sort examples by score s_i from largest to smallest.

Step 2: Sweep threshold t downward.

- Start with $t = +\infty$: predict all zeros $\implies (FPR, TPR) = (0, 0)$.
- Lower t so that more examples are predicted positive.
- Each time you pass an example:
 - ▶ If $y_i = 1$: TP increases $\implies$ TPR goes up.
 - ▶ If $y_i = 0$: FP increases $\implies$ FPR goes right.

Step 3: Plot the points $(FPR(t), TPR(t))$.

Result: A piecewise-linear, non-decreasing curve in the unit square.

Example: ROC by Threshold Sweep

Scores and labels:

Score	0.91	0.80	0.45	0.30	0.10
Label	1	0	1	0	0

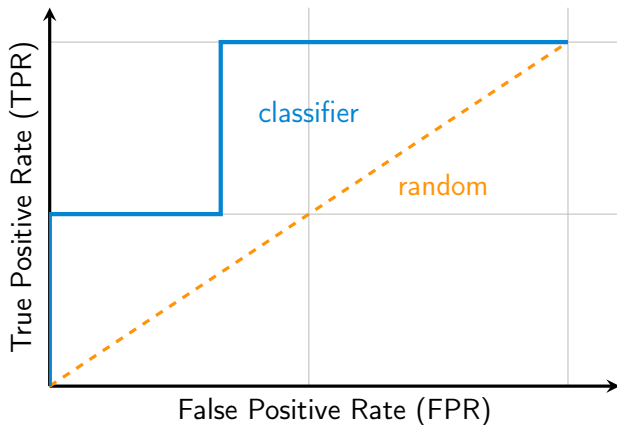
Sorted already from high to low.

Sweep thresholds:

Threshold t	Predictions	(TP,FN; FP,TN)	(FPR,TPR)
$t > 0.91$	all 0	(0,2; 0,3)	(0, 0)
$t = 0.91$	1st = 1	(1,1; 0,3)	(0, 0.5)
$t = 0.80$	1st 2 = 1	(1,1; 1,2)	(0.33, 0.5)
$t = 0.45$	first 3 = 1	(2,0; 1,2)	(0.33, 1)
$t = 0.30$	first 4 = 1	(2,0; 2,1)	(0.66, 1)
$t = 0.10$	all = 1	(2,0; 3,0)	(1, 1)

Plotting these points gives the ROC curve.

ROC Curve Visualization



A good classifier bends toward the upper-left corner (high TPR, low FPR). The diagonal is random guessing.

AUC: A Threshold-Independent Score

AUC = area under the ROC curve.

Interpretation:

- $AUC = 1$: perfect classifier.
- $AUC = 0.5$: random guessing.
- $AUC = \Pr(\text{random positive} > \text{random negative in score})$ (random guessing).

Why useful?

- Does not depend on any particular threshold.
- Robust to class imbalance.
- Measures how well scores *rank* positives above negatives.

In practice, ROC curves + AUC are standard for evaluating classifiers with probabilistic outputs (e.g. logistic regression).

What Is Probability Calibration?

A classifier outputs predicted probabilities $\hat{p}(x)$.

Calibration means:

$$\Pr(y = 1 \mid \hat{p}(X) \approx p) \approx p \quad \text{for all } p \in [0, 1].$$

Examples:

- Among all predictions near $\hat{p} = 0.2$, about 20% should actually be positive.
- Among all predictions near $\hat{p} = 0.8$, about 80% should be positive.

Well-calibrated models:

- logistic regression (usually),
- naive Bayes (poor),
- random forests (medium),
- deep nets (overconfident, often poorly calibrated).

Calibration is **separate** from accuracy.

How to Compute a Calibration Curve (Reliability Diagram)

Given predicted probabilities p_i and true labels y_i :

1. Split $[0, 1]$ into bins, e.g.

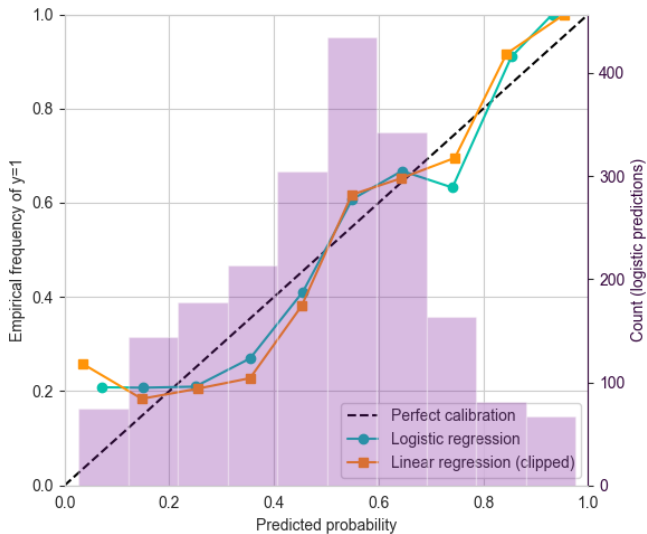
$$[0, 0.1), [0.1, 0.2), \dots, [0.9, 1.0].$$

2. For each bin B :

- ▶ $\hat{p}_B$ = average predicted probability in the bin,
- ▶ $\hat{y}_B$ = empirical fraction of positives in the bin.

3. Plot points $(\hat{p}_B, \hat{y}_B)$.

Calibration Curve Interpretation



Interpretation:

- **Perfect calibration:** points lie on the diagonal $y = x$.
- Above diagonal $\implies$ underconfident.
- Below diagonal $\implies$ overconfident.

Why Does Calibration Matter?

Decision-making depends on probabilities.

Examples:

- Medical diagnosis: $\hat{p} = 0.9$ should *really* mean 90% risk.
- Fraud detection: thresholding depends on calibrated scores.
- Multi-class classification: softmax outputs are often overconfident.

A classifier can have:

- high accuracy but poor calibration,
- good calibration but mediocre accuracy.

ROC curves measure ranking quality. Calibration measures probability quality.

Both are important, but different.

ROC Curves vs Calibration: When to Use Which?

Question	Use ROC / AUC	Use Calibration Curve
How well does the model <i>rank</i> positives above negatives?	Y: ROC/AUC measure ranking quality	
Predicted probabilities <i>meaningful</i> ?		Y: calibration measures probability accuracy
Class imbalance (rare positives)?	ROC/AUC are robust (insensitive) to imbalance	Calibration still works, but bins may need merging
Threshold selection (varying decision threshold)?	ROC ideal: shows tradeoff across all thresholds	Not designed for threshold choice

ROC Curves vs Calibration: When to Use Which?

Question	Use ROC / AUC	Use Calibration Curve
Decision-making under asymmetric costs (e.g. FP vs FN)?	ROC helps visualize tradeoffs	Calibration helps if decisions use probability estimates
Model comparison when only ranking matters (e.g. search, ranking tasks)?	ROC/AUC preferred	
Model comparison when downstream tasks use predicted probabilities (e.g. risk prediction)?		Calibration is essential
Neural networks (often overconfident)?	AUC may still look good	Calibration will reveal overconfidence

Summary: ROC curves evaluate *ranking*. Calibration curves evaluate *probability quality*. Both measure different, and complementary, aspects of model performance.

Cross Validation for Model Selection

Why Cross Validation?

Given data $\{(x_i, y_i)\}_{i=1}^n$, we want to estimate how well a model generalizes.

Problem: If we train and evaluate on the same data:

- training error is overly optimistic,
- overfitting looks good,
- we cannot compare models fairly.

Solution: Split the data into

- a set used for **training**, and
- a set used only for **validation**.

But a single split may be:

- unlucky,
- unstable (high variance),
- too small for training or validation.

Cross validation averages over many splits to stabilize the estimate.

Definition: k -Fold Cross Validation

k -fold cross validation works as follows:

1. Partition the dataset into k equal-sized folds:

$$D := \{(x_i, y_i)\}_{i=1}^n = F_1 \cup F_2 \cup \dots \cup F_k.$$

2. For each $j = 1, \dots, k$:

- ▶ Train on the other $k - 1$ folds:

Train on $D \setminus F_j$,

- ▶ Validate on fold F_j .

- ▶ Record the validation error E_j .

3. Compute metrics: e.g.,

$$\text{avg_error} = \frac{1}{k} \sum_{j=1}^k E_j$$

This gives a low-variance estimate of generalization performance.

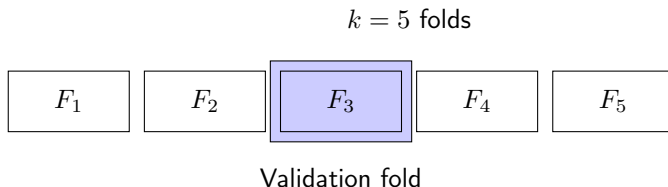
Other CV Metrics

$$\text{avg_error} = \frac{1}{k} \sum_{j=1}^k E_j \quad \text{and} \quad \text{std_error} = \sqrt{\frac{1}{k-1} \sum_{j=1}^k (E_j - \text{avg_error})^2}$$

$$\text{max_error} = \max_{j=1,\dots,k} E_j, \quad \text{min_error} = \min_{j=1,\dots,k} E_j,$$

$$\text{median_error} = \text{median}_{j=1,\dots,k} E_j.$$

Visualization of k -Fold Cross Validation



$$\text{Training} = F_1, F_2, F_4, F_5$$

In each round, we rotate the validation fold and retrain the model. After k rounds, we average the k validation errors.

Cross Validation for Model Selection

Suppose we want to choose amongst the following hyperparameters:

- regularization strength λ ,
- number of features p (or d),
- kernel bandwidth κ ,
- model class $\mathcal{C}$ (e.g. linear vs. polynomial vs. neural network).

For each candidate model $\mathcal{M}(\lambda, p, \kappa, \mathcal{C})$:

- run k -fold CV,
- compute its CV error `error_metric` ($\mathcal{M}(\lambda, p, \kappa, \mathcal{C})$).

Aside: Kernel Bandwidth (i.e. parameterization of a kernel)

In kernel methods (kernel regression, density estimation, RBF kernels), we use a kernel $K_{\kappa}(x, x_i)$ to measure the “similarity” between points. The parameter $\kappa = \text{bandwidth}$ controls how wide or narrow the kernel is.

Example (Gaussian kernel):

$$K_{\kappa}(x, x_i) = \exp\left(-\frac{\|x - x_i\|^2}{2\kappa^2}\right).$$

Interpretation of bandwidth:

- **Small** κ : kernels are narrow $\implies$ very local, high flexibility, can overfit
- **Large** κ : kernels are wide $\implies$ heavy smoothing, risk of underfitting

Cross Validation for Model Selection

Choose the model with the lowest CV error:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}(\lambda, p, \kappa, \mathcal{C})} \text{error_metric}(\mathcal{M}(\lambda, p, \kappa, \mathcal{C})).$$

The final model is then:

- retrained on the *full* dataset using $\mathcal{M}^*$,
- ready for test-time deployment.

Why Not Just One Train/Validation Split?

A single validation split has problems:

- **High variance:** the estimate depends on one partition.
- **Wasted data:** validation set not used for training.
- **Unfair to models:** one split may favor some models.
- **Unstable hyperparameters.**

Cross validation:

- uses *all* data for training and validation (on rotation),
- yields a more reliable estimate of generalization,
- reduces randomness from dataset splits,
- improves hyperparameter selection.

Choosing k and Practical Considerations

Common choices:

- $k = 5$ or 10 : good bias/variance tradeoff.
- $k = n$: **Leave-One-Out CV** (LOOCV) — high variance, costly.
- $k = 2$: simple split (train/test).

Practical notes:

- Shuffle data before splitting.
- Stratify folds for classification:

class proportions preserved in each fold.

- Use the **same folds** for all models being compared.
- After selecting hyperparameters, retrain on all data.

Cross validation is the gold standard for model selection in classical ML.

Extra Slides: Maximum Likelihood Estimation

From Probabilistic Modeling to Maximum Likelihood

In logistic regression, we model the conditional distribution:

$$p(y_i = 1 \mid x_i; \theta) = \sigma(\theta^\top x_i), \quad p(y_i = 0 \mid x_i; \theta) = 1 - \sigma(\theta^\top x_i),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$.

Assumption: Each label y_i is generated independently from a **Bernoulli** distribution:

$$y_i \sim \text{Bernoulli}(\sigma(\theta^\top x_i)).$$

Therefore, the likelihood is:

$$L(\theta) = \prod_{i=1}^n \left[\sigma(\theta^\top x_i) \right]^{y_i} \left[1 - \sigma(\theta^\top x_i) \right]^{1-y_i}.$$

This is the probability of observing the labels $\{y_i\}$ under the model.

What Is a Likelihood?

Suppose we have:

- data (observations) D ,
- a parametric model $p(D \mid \theta)$.

Definition (Likelihood). The *likelihood function* is

$$L(\theta) := p(D \mid \theta),$$

viewed as a function of the parameter θ with the data D fixed.

- Probability: a function of the **observed outcomes**.
- Likelihood: the **same quantity**, but viewed as a function of **parameters**.

Goal in Maximum Likelihood: Find parameter θ that makes the observed data most probable:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} L(\theta).$$

Likelihood Example: A Biased Coin

Suppose we flip a coin n times and see outcomes $y_1, \dots, y_n \in \{0, 1\}$.

Model:

$$y_i \sim \text{Bernoulli}(p), \quad p = \text{Pr}(\text{Heads}).$$

Probability of the whole dataset:

$$p(y_1, \dots, y_n \mid p) = \prod_{i=1}^n p^{y_i} (1-p)^{1-y_i}.$$

As a function of p , this is the **likelihood**:

$$L(p) = p^{\sum_i y_i} (1-p)^{n-\sum_i y_i}.$$

Interpretation: This tells us how plausible different values of p are, *given the data we observed*.

Likelihood in Supervised Learning

In supervised learning, we observe pairs (x_i, y_i) .

To define a likelihood, we must:

1. Choose a model for $p(y \mid x, \theta)$;
2. Assume the training examples are **independent**.

Then the joint probability of all labels given inputs $X = \{x_i\}$ and parameters θ is:

$$p(y_1, \dots, y_n \mid x_1, \dots, x_n, \theta) = \prod_{i=1}^n p(y_i \mid x_i, \theta).$$

This joint probability, treated as a function of θ , is the **likelihood**:

$$L(\theta) = \prod_{i=1}^n p(y_i \mid x_i, \theta).$$

Maximum likelihood picks θ making the observed labels most probable.

Likelihood for Logistic Regression

Model the conditional distribution:

$$p(y_i = 1 \mid x_i, \theta) = \sigma(\theta^\top x_i), \quad p(y_i = 0 \mid x_i, \theta) = 1 - \sigma(\theta^\top x_i).$$

Assumption: The labels $y_1, \dots, y_n$ are conditionally independent given the inputs.

Joint probability of the dataset:

$$p(y_1, \dots, y_n \mid x_1, \dots, x_n, \theta) = \prod_{i=1}^n \left[\sigma(\theta^\top x_i) \right]^{y_i} \left[1 - \sigma(\theta^\top x_i) \right]^{1-y_i}.$$

This is the likelihood:

$$L(\theta) = \prod_{i=1}^n \left[\sigma(\theta^\top x_i) \right]^{y_i} \left[1 - \sigma(\theta^\top x_i) \right]^{1-y_i}.$$

Maximizing this is equivalent to minimizing the logistic loss.